

A BUILD-BY-DOING TUTORIAL

FastAPI Zero to Production

From “what is an API?” to a deployed, tested, authenticated backend — one phase at a time.

7

PHASE SEVEN

Testing Your FastAPI App

Prove your API works automatically: pytest and TestClient, an isolated test database via dependency overrides, fixtures, testing CRUD, auth, and validation, plus coverage and continuous integration.

Contents

7.1 Why Testing Matters	4
The testing pyramid	5
7.2 FastAPI TestClient	5
7.3 Test Database Setup	6
7.4 Testing CRUD Endpoints	8
7.5 Testing Authentication	9
7.6 Testing Validation Errors	11
7.7 Test Coverage	11
What 80%+ means — and its limits	12
7.8 Continuous Integration Concept	12
Hands-On Exercise 7: Test the Book Library API	13
Checkpoint: At This Point You Can Now...	17
Phase 7 Summary	18

Your Journey

✓	Foundations: What Is an API?	DONE
✓	Setup & Your First API	DONE
✓	Path Params, Query Params & Body	DONE
✓	Pydantic: Data Validation	DONE
✓	Database Integration (SQLAlchemy)	DONE
✓	Authentication & Authorization	DONE
✓	Advanced FastAPI Features	DONE
7	Testing Your FastAPI App	YOU ARE HERE
8	Deployment with Docker	
★	Capstone — TaskFlow API	

Up to now you've tested your API by clicking around in `/docs`. That works for a quick check, but it's slow, manual, and you forget to re-test the old features when you add new ones. **Automated tests** fix this: you write code that exercises your API and asserts the results, then run it all in seconds with one command. The first time a test catches a bug you introduced *before* it reaches users, you'll be hooked. Testing is the single biggest confidence multiplier in software — let's build the habit.

7.1 Why Testing Matters

"If it's not tested, it's broken" sounds harsh, but it captures a real truth: code without tests is code you're *hoping* works. Every change risks silently breaking something else (a "regression"), and without tests you only find out when a user does. Tests turn hope into certainty, document how your code is meant to behave, and let you refactor fearlessly because you'll instantly know if you broke something.

The testing pyramid

Tests come in layers, usually drawn as a pyramid — many fast tests at the bottom, few slow ones at the top:

TEXT



End-to-End (few) — whole system, slowest, most realistic

Integration (some) — parts working together (API + DB)

Unit (many) — one function in isolation, fastest

- **Unit tests** check one small piece (a single function, a validator) in isolation. Fast and numerous.
- **Integration tests** check pieces working *together* — for an API, hitting an endpoint that touches the database. This is where most of *our* tests live.
- **End-to-end (E2E)** tests drive the whole system as a user would. Valuable but slow and brittle, so use sparingly.

For a FastAPI app, the sweet spot is plenty of **integration tests** that call real endpoints against a real (test) database, which is exactly what we'll write.

CONCEPT CHECK

Concept Check: Calling `POST /books` in a test and asserting the book then appears in `GET /books` — which layer is that? *Answer: An integration test. It exercises the route, the validation, and the database together, rather than one isolated function.*

7.2 FastAPI TestClient

FastAPI's `TestClient` lets you call your app **in-process** — no need to start a real server. It sends requests straight to your app and returns the responses, fast and reliably.

BASH / TERMINAL

```
# httpx powers the TestClient; pytest is the test runner.
pip install pytest httpx
```

PYTHON

```
# test_main.py
from fastapi.testclient import TestClient
from main import app          # import your FastAPI app

client = TestClient(app)

# pytest discovers any function named test_*
def test_hello():
    response = client.get("/hello")          # call the endpoint
    assert response.status_code == 200        # check the status
    assert response.json() == {"message": "Hello, World!"} # check the body
```

Run all tests with one command:

BASH / TERMINAL

```
# pytest finds and runs every test_*.py file and test_* function.
pytest
```

`TestClient` works because FastAPI apps speak **ASGI**, and the client talks that protocol directly — so there's no network, no port, no separate process. Tests run in milliseconds.

TIP

Tip — anatomy of a test: Most tests follow **Arrange → Act → Assert**: set up the data you need (Arrange), call the thing you're testing (Act), then check the result (Assert). Keeping that shape makes tests easy to read and to fix when they fail.

7.3 Test Database Setup

You must **never** run tests against your real database — they'd create and delete data for real. Instead, point the app at a separate, disposable test database, and the clean way to do that is FastAPI's **dependency override**: swap `get_db` for a version that uses the test database.

PYTHON

```

# conftest.py – shared pytest fixtures, auto-discovered by pytest
import pytest
from fastapi.testclient import TestClient
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from app.database import Base, get_db
from app.main import app

# A separate SQLite database file just for tests.
TEST_URL = "sqlite:///./test.db"
engine = create_engine(TEST_URL, connect_args={"check_same_thread": False})
TestingSessionLocal = sessionmaker(bind=engine, autoflush=False)

@pytest.fixture
def db_session():
    # Fresh schema for each test: create tables, yield a session, then drop.
    Base.metadata.create_all(bind=engine)
    session = TestingSessionLocal()
    try:
        yield session
    finally:
        session.close()
        Base.metadata.drop_all(bind=engine)  # clean slate every test

@pytest.fixture
def client(db_session):
    # Override get_db so the app uses the TEST database during this test.
    def override_get_db():
        try:
            yield db_session
        finally:
            pass
    app.dependency_overrides[get_db] = override_get_db
    yield TestClient(app)
    app.dependency_overrides.clear()  # remove the override after

```

The key line is `app.dependency_overrides[get_db] = override_get_db`. Because every route gets its database through `Depends(get_db)`, replacing that one dependency redirects the *entire app* to the test database — no route code changes. This is the payoff of the dependency injection you set up in Phase 4.

TIP

Tip — what is `conftest.py`? It's a special pytest file for shared **fixtures**. A *fixture* is reusable setup code; any test (or fixture) can request it just by naming it as an argument. Put fixtures in `conftest.py` and pytest makes them available to every test file automatically — no imports needed.

COMMON MISTAKES

Common Mistakes: Letting tests share state. If one test creates a book and the next test assumes an empty database, they're coupled and will fail in confusing ways. The `create_all / drop_all` per test (above) gives each test a **clean slate**, so tests are independent and can run in any order.

7.4 Testing CRUD Endpoints

Now test the full lifecycle of a resource. Each test asks for the `client` fixture, which hands over a `TestClient` wired to the test database.

PYTHON

```
def test_create_book(client):
    # Act: create a book.
    resp = client.post("/books", json={"title": "Dune", "author": "Herbert"})
    # Assert: correct status and echoed data, plus a server-assigned id.
    assert resp.status_code == 201
    data = resp.json()
    assert data["title"] == "Dune"
    assert "id" in data

def test_get_book(client):
    # Arrange: create one first.
    created = client.post("/books", json={"title": "1984", "author": "Orwell"}).json()
    # Act + Assert: fetch it back and check it matches.
    resp = client.get(f"/books/{created['id']}")
    assert resp.status_code == 200
    assert resp.json()["title"] == "1984"

def test_update_book(client):
    created = client.post("/books", json={"title": "Old", "author": "A"}).json()
    resp = client.put(f"/books/{created['id']}",
                     json={"title": "New", "author": "A"})
    assert resp.status_code == 200
    assert resp.json()["title"] == "New"

def test_delete_book(client):
    created = client.post("/books", json={"title": "Temp", "author": "A"}).json()
    assert client.delete(f"/books/{created['id']}").status_code == 204
    # Verify it's really gone.
    assert client.get(f"/books/{created['id']}").status_code == 404

def test_get_missing_book_returns_404(client):
    # The unhappy path: a resource that doesn't exist.
    assert client.get("/books/99999").status_code == 404
```

Notice the rhythm: test the **happy path** (it works) *and* the **unhappy path** (it fails correctly, like the `404`). Both matter — a `404` that's actually a `500` is a real bug.

CONCEPT CHECK

Concept Check: Why assert on the **status code** *and* the **body**, not just one? *Answer: They can disagree. An endpoint might return the right data but the wrong status (e.g. `200` instead of `201`), or the right status with a wrong/missing field. Checking both catches more bugs.*

7.5 Testing Authentication

Auth flows need their own tests: registration (including duplicates), login (right and wrong credentials), and protected routes (with and without a token). A small helper keeps token-handling tidy.

PYTHON

```

def test_register_success(client):
    resp = client.post("/auth/register",
                       json={"username": "ada", "password": "password123"})
    assert resp.status_code == 201
    assert "password" not in resp.json()      # never leak the password

def test_register_duplicate(client):
    body = {"username": "ada", "password": "password123"}
    client.post("/auth/register", json=body)
    # Second time should be rejected.
    assert client.post("/auth/register", json=body).status_code == 400

def test_login_wrong_password(client):
    client.post("/auth/register",
               json={"username": "ada", "password": "password123"})
    resp = client.post("/auth/token",
                      data={"username": "ada", "password": "WRONG"})
    assert resp.status_code == 401

def auth_header(client, username="ada", password="password123"):
    # Helper: register + log in, return a ready-to-use Authorization header.
    client.post("/auth/register",
               json={"username": username, "password": password})
    token = client.post("/auth/token",
                      data={"username": username, "password": password}).json()["access_
ken"]
    return {"Authorization": f"Bearer {token}"}

def test_protected_requires_token(client):
    # No token → 401.
    assert client.post("/books", json={"title": "X"}).status_code == 401

def test_protected_with_token(client):
    # With a valid token → success.
    resp = client.post("/books", json={"title": "X"},
                      headers=auth_header(client))
    assert resp.status_code == 201

```

TIP

Tip: Turn `auth_header` into a **fixture** (e.g. `authenticated_client`) when many tests need a logged-in client — pytest will inject it wherever you name it. Fixtures compose: a fixture can request other fixtures, so an `authenticated_client` fixture can build on the `client` fixture.

7.6 Testing Validation Errors

Your Pydantic rules from Phase 3 deserve tests too. Send deliberately bad data and assert a `422`, and check the error points at the right field.

PYTHON

```
def test_missing_required_field(client):
    # Omit a required field → 422.
    resp = client.post("/books", json={"author": "Herbert"}) # no title
    assert resp.status_code == 422

def test_invalid_type(client):
    # Wrong type for year → 422.
    resp = client.post("/books",
                        json={"title": "X", "author": "Y", "year": "soon"})
    assert resp.status_code == 422

def test_validation_error_mentions_field(client):
    resp = client.post("/books", json={"author": "Herbert"})
    body = resp.json()
    # The 422 body lists each problem with a `loc` path.
    locations = [err["loc"] for err in body["detail"]]
    assert any("title" in loc for loc in locations)
```

COMMON MISTAKES

Common Mistakes: Testing only valid input. The whole point of validation is to reject bad input — so the tests that *send* bad input and assert a `422` are the ones that actually prove your validation works. Untested validation is unproven validation.

7.7 Test Coverage

Coverage measures how much of your code your tests actually execute. The `pytest-cov` plugin reports it.

BASH / TERMINAL

```
pip install pytest-cov

# Run tests and report coverage for the app package; also write an HTML report.
pytest --cov=app --cov-report=term --cov-report=html
```

The terminal shows a percentage per file; the HTML report (`htmlcov/index.html`) highlights exactly which lines were never run — invaluable for spotting untested branches.

TEXT

Name	Stmts	Miss	Cover
-----	-----	-----	-----
app/main.py	12	0	100%
app/crud.py	34	3	91%
app/routers/books.py	40	5	88%
-----	-----	-----	-----
TOTAL	86	8	91%

What 80%+ means — and its limits

Aiming for **80%+ coverage** is a sensible, common target: it means the large majority of your code runs under test. But coverage measures *quantity*, *not quality* — it tells you a line *ran*, not that you *asserted the right thing* about it. You can have 100% coverage and still miss bugs if your assertions are weak. Treat coverage as a guide to find untested areas, not as proof of correctness.

TIP

Tip: Don't chase 100%. The last few percent often covers trivial or defensive code and costs more effort than it's worth. Focus coverage on your **business logic and error paths** — the places bugs actually hurt. 80% of meaningful code beats 100% of everything.

7.8 Continuous Integration Concept

Continuous Integration (CI) means your tests run automatically on a server every time you push code — so a teammate's broken change is caught before it's merged, not after it ships. **GitHub Actions** is a popular, free way to do this. You add a YAML file describing the steps:

PYTHON

```
# .github/workflows/tests.yml (YAML, shown here as text)
# name: Tests
# on: [push, pull_request]      # run on every push and PR
# jobs:
#   test:
#     runs-on: ubuntu-latest
#     steps:
#       - uses: actions/checkout@v4
#       - uses: actions/setup-python@v5
#         with: { python-version: "3.11" }
#       - run: pip install -r requirements.txt
#       - run: pytest --cov=app
```

On every push, GitHub spins up a fresh machine, installs your dependencies, and runs `pytest`. If any test fails, the push is flagged with a red and the team is notified.

Why CI is non-negotiable on real teams: it guarantees that every change is tested the same way, on a clean machine, with no "but it worked on my laptop." It's the safety net that lets many people work on one codebase without constantly breaking each other.

CONCEPT CHECK

Concept Check: Why run tests on a CI server instead of trusting everyone to run them locally? *Answer: People forget, skip, or have different setups. CI runs the same tests on a clean machine for every change automatically — consistent, unskippable, and visible to the whole team.*

HANDS-ON

Hands-On Exercise 7: Test the Book Library API

Write a full test suite for a secured Book Library: a test-database fixture, a client fixture with `get_db` overridden, an auth-header helper, and tests covering CRUD, auth, ownership, and validation — both happy and unhappy paths. Below is a complete, runnable suite (app + tests in one place for clarity; in a real project the app lives in `app/` and tests in `tests/`).

```

# conftest.py-style fixtures + tests (run with: pytest test_books.py)
import pytest
from fastapi.testclient import TestClient
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
# from app.main import app
# from app.database import Base, get_db
from book_app import app, Base, get_db, engine_for_tests  # your app module

@pytest.fixture
def client():
    # Fresh in-memory DB per test via a StaticPool engine.
    engine = engine_for_tests()
    TestingSession = sessionmaker(bind=engine, autoflush=False)
    Base.metadata.create_all(bind=engine)

    def override_get_db():
        db = TestingSession()
        try:
            yield db
        finally:
            db.close()

    app.dependency_overrides[get_db] = override_get_db
    yield TestClient(app)
    app.dependency_overrides.clear()

def auth_header(client, username="ada", password="password123"):
    client.post("/auth/register",
                json={"username": username, "password": password})
    token = client.post("/auth/token",
                        data={"username": username, "password": password}).json()["access_to
ken"]
    return {"Authorization": f"Bearer {token}"}

# ---- auth tests ----
def test_register_success(client):
    r = client.post("/auth/register",
                    json={"username": "ada", "password": "password123"})
    assert r.status_code == 201 and "password" not in r.json()

def test_register_duplicate(client):
    body = {"username": "ada", "password": "password123"}
    client.post("/auth/register", json=body)
    assert client.post("/auth/register", json=body).status_code == 400

def test_register_short_password(client):
    r = client.post("/auth/register", json={"username": "ada", "password": "x"})
    assert r.status_code == 422

def test_login_success(client):
    auth_header(client)  # registers + logs in without error

```

```

def test_login_wrong_password(client):
    client.post("/auth/register",
                json={"username": "ada", "password": "password123"})
    assert client.post("/auth/token",
                        data={"username": "ada", "password": "no"}).status_code == 401

def test_login_unknown_user(client):
    assert client.post("/auth/token",
                        data={"username": "ghost", "password": "x"}).status_code == 401

# ---- crud + ownership tests ----
def test_list_books_public(client):
    assert client.get("/books").status_code == 200

def test_create_requires_auth(client):
    assert client.post("/books", json={"title": "Dune"}).status_code == 401

def test_create_book(client):
    r = client.post("/books", json={"title": "Dune"}, headers=auth_header(client))
    assert r.status_code == 201 and r.json()["title"] == "Dune"

def test_create_then_list(client):
    h = auth_header(client)
    client.post("/books", json={"title": "Dune"}, headers=h)
    client.post("/books", json={"title": "1984"}, headers=h)
    assert len(client.get("/books").json()) == 2

def test_get_one_book(client):
    h = auth_header(client)
    bid = client.post("/books", json={"title": "Dune"}, headers=h).json()["id"]
    assert client.get(f"/books/{bid}").json()["title"] == "Dune"

def test_get_missing_book_404(client):
    assert client.get("/books/9999").status_code == 404

def test_delete_own_book(client):
    h = auth_header(client)
    bid = client.post("/books", json={"title": "Dune"}, headers=h).json()["id"]
    assert client.delete(f"/books/{bid}", headers=h).status_code == 204
    assert client.get(f"/books/{bid}").status_code == 404

def test_delete_others_book_forbidden(client):
    owner = auth_header(client, "ada")
    bid = client.post("/books", json={"title": "Dune"}, headers=owner).json()["id"]
    other = auth_header(client, "bob")
    assert client.delete(f"/books/{bid}", headers=other).status_code == 403

def test_delete_missing_book_404(client):
    h = auth_header(client)
    assert client.delete("/books/9999", headers=h).status_code == 404

# ---- validation tests ----
def test_create_missing_title(client):
    assert client.post("/books", json={}, headers=auth_header(client)).status_code == 422

def test_create_empty_title(client):
    assert client.post("/books", json={"title": ""},

```

```

        headers=auth_header(client)).status_code == 422

def test_validation_error_points_to_field(client):
    r = client.post("/books", json={}, headers=auth_header(client))
    locs = [e["loc"] for e in r.json()["detail"]]
    assert any("title" in loc for loc in locs)

def test_invalid_token_rejected(client):
    r = client.post("/books", json={"title": "X"},
                    headers={"Authorization": "Bearer garbage"})
    assert r.status_code == 401

```

Run the suite:

BASH / TERMINAL

```

# Run all tests, see a short summary, and measure coverage.
pytest -v --cov=book_app --cov-report=term

```

Expected: every test passes (a row of green dots), and coverage of your app code lands comfortably above 80%.

What just happened? You built a real safety net. The `client` fixture spins up a **fresh, isolated database for every test** and overrides `get_db` so the whole app talks to it — no test ever touches real data, and tests can't interfere with each other. The `auth_header` helper makes testing protected and owner-only routes trivial. The suite covers the **happy paths** (register, login, create, list, get, delete-own) *and* the **unhappy paths** (duplicate registration, wrong password, missing token, other-user delete [403](#), missing resource [404](#), and validation [422](#)s) — over twenty tests in all. Run `pytest` and in a second you know your entire API still works. That confidence is what lets you keep changing code without fear.

COMMON MISTAKES

Common Mistakes: Forgetting to clear `app.dependency_overrides` after a test, leaking the override into other tests. The `yield ... ; app.dependency_overrides.clear()` pattern in the fixture guarantees cleanup even if a test fails.

TIP

Tip: Run `pytest` constantly — after every small change, not just at the end. A failing test the moment you introduce a bug is a tiny, easy fix; a bug discovered days later, buried under more changes, is a painful hunt.

Checkpoint: At This Point You Can Now...

- ☐ Explain **why tests matter** and place tests on the **testing pyramid** (unit / integration / E2E).
 - ☐ Use FastAPI's `TestClient` to call endpoints in-process and assert status + body.
 - ☐ Set up an **isolated test database** and override `get_db` with `app.dependency_overrides`.
 - ☐ Write **fixtures** in `conftest.py`, including a `client` fixture with a clean slate per test.
 - ☐ Test **CRUD** endpoints on both happy and unhappy paths (including `404`).
 - ☐ Test **authentication**: registration, login, and protected/owner-only routes with an auth helper.
 - ☐ Test **validation errors** (`422`) and check the error points at the right field.
 - ☐ Measure **coverage** with `pytest-cov`, interpret it, and explain its limits.
 - ☐ Describe **CI** and why it runs tests automatically on every push.
-

Phase 7 Summary

PHASE 7 • KEY TAKEAWAYS

In this phase you turned hope into certainty with an automated test suite.

- Untested code is code you're *hoping* works; tests catch **regressions**, document behaviour, and enable fearless refactoring. The **pyramid**: many unit, some integration, few E2E — for APIs, lean on **integration** tests.
- **TestClient** calls your app in-process over ASGI (no server, no network); pytest discovers `test *` functions; assert **status code and body**.
- Use a **separate test database** and `app.dependency_overrides[get_db]` to redirect the whole app — the payoff of dependency injection. Give each test a **clean slate** (`create_all` / `drop_all`).
- **Fixtures** in `conftest.py` provide reusable setup (`client`, `db_session`, `authenticated_client`) and compose with one another.
- Test **CRUD**, **auth** (register/login/protected/owner), and **validation 422s** — always covering the **unhappy paths**, not just the happy ones.
- **pytest-cov** measures coverage; aim for **80%+** of meaningful code, but remember coverage is quantity, not quality.
- **CI** (e.g. GitHub Actions) runs your tests automatically on every push — consistent, unskippable, team-wide.
- **You wrote a 20+ test suite** for the Book Library with isolated DB fixtures, an auth helper, and full happy/unhappy coverage.

Your API is now provably correct. Next phase, you'll ship it to the world with Docker.

End of Phase 7. Reply "**continue**" for Phase 8 — Deployment with Docker — where you containerise the app and deploy it live.